

 UNIFACS	Projeto A3 UC Teoria da Computação e compiladores	2026.1
Professor	Inácio Borges	

Projeto A3 – Diretrizes

PROJETO FINAL: CONSTRUÇÃO DE UM MINI-COMPILADOR (FRONT-END E SEMÂNTICA BÁSICA)

Objetivo do Projeto

O objetivo deste projeto final é consolidar os conhecimentos teóricos e práticos da disciplina de Teoria da Computação e Compiladores através do desenvolvimento de um software real.

Vocês irão construir as fases iniciais de um compilador: o Analisador Léxico (Scanner), o Analisador Sintático (Parser) com geração de Árvore (AST) e um Analisador Semântico. Cada equipe ficará responsável por implementar este compilador reduzido para uma "Mini-Linguagem" específica.

Nota: O seu compilador não precisa gerar código de máquina executável. O objetivo do sistema é ler um arquivo de texto contendo o código-fonte, analisar a estrutura, reportar erros e exibir as estruturas internas geradas.

DATA PRÉVIA: 20/05

ENTREGA FINAL: 27/05

EXPO UNIFACS: 10/06

 UNIFACS	Projeto A3 UC Teoria da Computação e compiladores	2026.1
Professor	Inácio Borges	

O Escopo Obrigatório (Requisitos Funcionais)

O compilador deve ser capaz de reconhecer, validar e estruturar as seguintes operações básicas para TODAS as linguagens:

1. **Atribuição:** Atribuir valores numéricos (inteiros/decimais) ou strings a variáveis.
2. **Matemática:** Respeitar a precedência matemática das operações de Soma (+), Subtração (-), Multiplicação (*), Divisão (/) e o uso de parênteses ().
3. **Controle de Fluxo:** Suportar condições if (com else opcional) e laços while.
4. **Relacionais:** Maior (>), Menor (<), Igualdade (==), Diferença (!=).

Distribuição das Equipes e Especificações Detalhadas

Atenção aos detalhes: as regras abaixo são estritas. O compilador de vocês deve rejeitar códigos que violem estas regras específicas, mesmo que pareçam corretos em outras linguagens.

 UNIFACS	Projeto A3 UC Teoria da Computação e compiladores	2026.1
Professor	Inácio Borges	

Tema 1: Mini-C

Equipe:

Gustavo do Carmo Silva 1272322619

Enzo Santos 12724121802

Patrick Ryan 1272322735

Kaique Filipus 12724137376

Linguagem de tipagem estática rigorosa.

- **Tipos Suportados:** Apenas int, float, e void.
- **Declaração de Variáveis:** O tipo deve preceder o identificador. Pode haver declaração simples (int x;) ou com atribuição imediata (int x = 10;).
- **Operadores Lógicos:** Exclusivamente &&, ||, e !.
- **Delimitadores de Instrução:** O ponto e vírgula (;) é estritamente obrigatório ao final de declarações, atribuições e chamadas de função. Se faltar o ;, o Sintático deve acusar erro.
- **Blocos de Escopo:** O corpo das funções, do if, do else e do while deve ser **obrigatoriamente** envolvido por chaves { }.
- **Assinatura de Funções:** Deve indicar o tipo de retorno, nome, e parâmetros tipados.

Exemplo:

```
int calcular(int base) {
    int resultado = base * 2;
    while (resultado > 0) {
        resultado = resultado - 1;
    }
    return resultado;
}
```

 UNIFACS	Projeto A3 UC Teoria da Computação e compiladores	2026.1
Professor	Inácio Borges	

Tema 2: Mini-Java

Equipe:

Irvin Marques de Moura 12723123617

Gabriel Moreira de Oliveira 1272311981

Davi Jansen Almeida de Souza 1272326233

Artur Jansen Almeida de Souza 1272326228

Linguagem orientada a objetos de tipagem estática (focada na estrutura de classes).

- **Tipos Suportados:** Apenas int, float, boolean e String.
- **Estrutura Obrigatória (Classe):** Todo o código, incluindo variáveis globais e funções (métodos), deve obrigatoriamente estar dentro de um bloco de classe. O Sintático deve esperar o token class no início do arquivo. Não é necessário exigir modificadores (public, static).
- **Saída de Dados:** O sistema deve reconhecer a chamada específica System.out.println(expressao); como um comando nativo do sistema.
- **Operadores Lógicos:** Exclusivamente &&, ||, e !.
- **Delimitadores e Blocos:** Ponto e vírgula (;) obrigatório. Escopos com chaves { }.
- **Exemplo:**

```
class Main {
    int valor = 5;

    int dobrar(int x) {
        return x * 2;
    }

    if (valor == 5) {
        System.out.println(dobrar(valor));
    }
}
```

 UNIFACS	Projeto A3 UC Teoria da Computação e compiladores	2026.1
Professor	Inácio Borges	

Tema 3: Mini-JavaScript

Equipe - 3

Lino Menezes de Castro 12723114033

Johan Victor Prazeres Santos 1272312828

Morgana dos Santos Martins 12723116583

João Vítor Costa Sousa 12723122280

Linguagem procedural dinâmica com flexibilidade de delimitadores.

- **Declaração de Variáveis:** O tipo numérico/string é inferido. A declaração deve usar estritamente as palavras-chave `let` ou `const`. O uso de `var` ou declaração implícita (ex: `x = 10` sem `let` antes) deve gerar um **Erro Sintático**.
- **Operadores Lógicos:** Exclusivamente `&&`, `||`, e `!`.
- **Delimitadores de Instrução (O Desafio Léxico):** O ponto e vírgula (`;`) é **opcional**. O Analisador Léxico deve ser programado para emitir um token de fim de comando (como `FIM_INSTRUCAO`) sempre que encontrar um `;` OU uma quebra de linha (`\n`) válida após uma expressão.
- **Assinatura de Funções:** Usam a palavra-chave `function` sem indicar tipo de retorno.
- **Exemplo:**

```
const taxa = 1.5
let contador = 0; // Ponto e vírgula opcional
```

```
function somar(a, b) {
  let resultado = a + b
  return resultado * taxa
}
```

 UNIFACS	Projeto A3 UC Teoria da Computação e compiladores	2026.1
Professor	Inácio Borges	

Tema 4: Mini-PHP

Equipe

Ruan Cláudio Damasceno Souza 12725169864

Arthur Souza Freire 12724128026

Miqueias Rangel dos Santos Almeida 12724122517

Mário Jorge Figueiredo de Oliveira Neto 1272419829

Linguagem dinâmica com regras estritas de nomenclatura de identificadores.

- **Declaração de Variáveis (O Desafio Léxico):** Não existem palavras-chave como int ou let. A tipagem é dinâmica, mas **todo e qualquer identificador de variável deve obrigatoriamente começar com o símbolo de cifrão (\$).**
 - Regra Regex: `^[a-zA-Z_][a-zA-Z0-9_]*`. Se o utilizador escrever valor = 10;, o Scanner deve gerar um erro dizendo que o identificador é inválido. O correto é \$valor = 10;.
- **Operadores Lógicos:** Exclusivamente &&, ||, e !.
- **Delimitadores e Blocos:** Ponto e vírgula (;) obrigatório em todas as instruções simples. Escopos definidos por chaves { }.
- **Assinatura de Funções:** O nome da função *não* leva \$, mas os parâmetros levam.
- **Exemplo:**

```
$numero = 10;
$limite = 20;
```

```
function processar($valor) {
    if ($valor < 100) {
        $valor = $valor * 2;
    }
    return $valor;
}
```

 UNIFACS	Projeto A3 UC Teoria da Computação e compiladores	2026.1
Professor	Inácio Borges	

Tema 5: Mini-Python

Equipe

Kauan de Almeida Santos 1272418213

Daniel Cerqueira Gomes 1272417325

Davi Vieira Falcão 12724148200

Kelvin Oliveira dos Santos 12724122807

Linguagem dinâmica estruturada unicamente por espaços em branco (indentação).

- **Declaração de Variáveis:** Atribuição totalmente direta. Sem int, sem let, sem \$. Exemplo: `x = 10`.
- **Operadores Lógicos:** Proibido o uso de `&&` e `||`. Deve usar exclusivamente as palavras-chave `and`, `or`, e `not`.
- **Delimitadores de Instrução:** Proibido o uso de ponto e vírgula (`;`). O fim da instrução é sempre a quebra de linha (`\n`). As assinaturas de `if`, `else`, `while` e `def` devem terminar obrigatoriamente com dois pontos (`:`).
- **Blocos de Escopo (O Grande Desafio):** A linguagem não usa chaves `{ }`. O escopo é estritamente definido pela indentação (espaços à esquerda).
 - **Regra para o Léxico:** O Analisador Léxico deve manter uma pilha (stack) contando a quantidade de espaços no início de cada linha. Ao detectar um aumento de espaços, deve gerar um token artificial chamado `INDENT`. Ao detectar uma diminuição, deve gerar um token artificial chamado `DEDENT`. O Parser usará estes tokens no lugar das chaves.
- **Exemplo:**

```
def calcular(x, y):
    resultado = x + y
    while resultado > 0:
        resultado = resultado - 1
    return resultado
```

 UNIFACS	Projeto A3 UC Teoria da Computação e compiladores	2026.1
Professor	Inácio Borges	

Tema 6: Kotlin

Equipe

Bruno Melo 12725138592

Daniel Meneses 127244138845

Fernando Mota 127244138845

Mini-Kotlin

Linguagem estática (ou de tipagem inferida) estruturada unicamente por **espaços em branco (indentação)**, eliminando a necessidade de chaves para definir blocos.

- **Declaração de Variáveis:** Uso obrigatório de `val` (imutável) ou `var` (mutável), mas com inferência de tipo total. Exemplo: `val x = 10`.
- **Operadores Lógicos:** Proibido o uso de `&&`, `||` e `!`. Devem ser substituídos pelas palavras-chave `and`, `or` e `not` (simulando funções infixas ou operadores legíveis).
- **Delimitadores de Instrução:** O fim da instrução é sempre a quebra de linha (`\n`).
- **Blocos de Escopo:** A linguagem **não utiliza chaves** `{ }`. O escopo de funções (`fun`), condicionais (`if`) e laços (`while`, `for`) é estritamente definido pela indentação.
- **Regra para o Léxico:** O Analisador Léxico deve manter uma **pilha (stack)** para rastrear o nível de profundidade da indentação:
 - Ao aumentar o recuo, o Léxico emite o token **INDENT**.
 - Ao diminuir o recuo, o Léxico emite o token **DEDENT**.
 - O Parser utiliza esses tokens para delimitar onde começariam e terminariam as chaves no Kotlin tradicional.

Exemplo de Código:

```
fun calcular(x: Int, y: Int)
    var resultado = x + y
    while resultado > 0
        println(resultado)
```


 UNIFACS	Projeto A3 UC Teoria da Computação e compiladores	2026.1
Professor	Inácio Borges	

resultado = resultado - 1

return resultado

Tema 7: Mini-Ruby

Equipe

Lucas Lopes Albino 12723133422

Lucas Ribeiro de Souza Neves 12723130991

Kaio Breno de Freitas Pinho 12724130159

Luiz Henrique Figueredo Santos 1272417293

Linguagem dinâmica estruturada unicamente por **espaços em branco (indentação)**, eliminando a necessidade da palavra-chave end.

- **Declaração de Variáveis:** Atribuição direta. Sem tipos ou prefixos desnecessários. Exemplo: `x = 10`.
- **Operadores Lógicos:** Proibido o uso de `&&`, `|` e `!`. Deve usar exclusivamente as palavras-chave nativas do Ruby: `and`, `or` e `not`.
- **Delimitadores de Instrução:** Proibido o uso de ponto e vírgula (`;`). O fim da instrução é sempre a quebra de linha (`\n`).
- **Blocos de Escopo (O Grande Desafio):** A linguagem **não utiliza** a palavra-chave `end` para fechar métodos, condicionais ou laços. O escopo é estritamente definido pela indentação (espaços à esquerda).
- **Regra para o Léxico:** O Analisador Léxico deve manter uma **pilha (stack)** controlando a quantidade de espaços no início de cada linha:
 - Ao detectar um aumento no número de espaços em relação ao topo da pilha, gera o token **INDENT**.
 - Ao detectar uma diminuição, gera o token **DEDENT**.
 - O Parser utilizará esses tokens para identificar o fechamento de blocos que, no Ruby padrão, seriam encerrados por `end`.

 UNIFACS	Projeto A3 UC Teoria da Computação e compiladores	2026.1
Professor	Inácio Borges	

Exemplo de Código:

```
def calcular(x, y)
    resultado = x + y
    while resultado > 0
        puts resultado
        resultado = resultado - 1
    return resultado
```

 UNIFACS	Projeto A3 UC Teoria da Computação e compiladores	2026.1
Professor	Inácio Borges	

Fases do Projeto, Entregáveis e Divisão de Trabalho

Fase 1: Modelagem Matemática e Documentação

Antes de programar, a equipa deve modelar a linguagem. Entregáveis:

- **Tabela de Tokens Léxicos:** Mapeamento exato de todos os tokens (com as Regex correspondentes).
- **Gramática Livre de Contexto (GLC):** O conjunto de regras de produção ($S \rightarrow \dots$). A gramática não pode ser ambígua (deve forçar a precedência da multiplicação sobre a soma criando as camadas Expressão, Termo, Fator).

Fase 2: O Analisador Léxico / Scanner

- **A Tarefa:** Consumir o arquivo de texto, aplicar as Regex definidas na Fase 1 e gerar a fita de Tokens.
- **Tratamento Léxico:** Descartar espaços e comentários apropriadamente
- **Tratamento de Erros:** O léxico deve interromper a análise ao encontrar um caractere estranho ao alfabeto da linguagem, informando a linha e a coluna.

Fase 3: Analisador Sintático e Tratamento de Erros

- **A Tarefa:** Consumir o fluxo de Tokens e validar a ordem lógica segundo a GLC. Pode ser implementado manualmente (Descida Recursiva) ou via ferramentas.
- **Mensagens de Erro Inteligentes:** Se um utilizador não fechar uma chave, a mensagem não pode ser "Erro fatal". Deve informar: *Erro Sintático (Linha X): Esperado '}', encontrado '}'*. O Parser deve tentar "recuperar" o erro (pular o símbolo problemático) para encontrar mais erros no resto do ficheiro, em vez de falhar no primeiro problema.

Fase 4: Geração da AST - Abstract Syntax Tree

- **A Tarefa:** O Sintático deve construir em memória uma árvore representando as operações lógicas.
- **Saída Visual:** Ao concluir, deve ser impresso na tela (ou exportado em JSON) o desenho da árvore. A árvore deve demonstrar claramente que um nó While tem um filho *Condicao* e um filho *BlocoDeComandos*.

Fase 5: Tabela de Símbolos e Semântica Básica

- **A Tarefa:** Validar o significado das estruturas durante a passagem pela árvore.

 UNIFACS	Projeto A3 UC Teoria da Computação e compiladores	2026.1
Professor	Inácio Borges	

- **Regras Semânticas:**

1. Guardar o nome, escopo e tipo das variáveis em um Hash Map.
2. Emitir "Erro Semântico" se uma variável for usada em uma expressão matemática sem ter sido declarada antes.
3. Emitir "Erro Semântico" se houver uma tentativa de redeclarar a mesma variável no mesmo escopo (ex: `int x = 5; int x = 10;`).

 UNIFACS	Projeto A3 UC Teoria da Computação e compiladores	2026.1
	Professor Inácio Borges	

Critérios de Avaliação

O projeto será avaliado com base nos seguintes critérios:

Critério	Descrição Detalhada	Peso
Modelagem Formal	GLC robusta, sem ambiguidades e Expressões Regulares abrangentes.	15%
Scanner (Léxico)	Reconhecimento de lexemas, descarte de comentários e tratamento de regras específicas da linguagem (ex: \$, indentação, ; opcional).	15%
Parser (Sintático)	Consumo preciso da fita de tokens e respeito rigoroso à GLC.	25%
Mensagens de Erro	Capacidade de reportar linha/coluna exatas e tentar recuperação de erros.	15%
AST e Semântica	Qualidade estrutural da árvore gerada (Tabela de Símbolos).	20%
Apresentação	Demonstração prática do compilador rodando algoritmos de sucesso e falha elaborados no momento da apresentação.	10%